ITM SKILLS UNIVERSITY

School Of Future Tech'29

CASE STUDY - 154 Report

Data Structure & Algorithms

(Prof. Aarti Pardeshi)

Title- **Smart ride Allocation System**

Name- Stuti Saxena

Cohort- Larry Page

Roll number- 150096725024

Batch- 2025-2029

Links-

Project's github-

https://github.com/stutisaxena44/DSA-Smart-Ride-Allocation

1. PROBLEM STATEMENT

Ride-sharing companies operate in highly dynamic environments where both riders and drivers are constantly changing locations. At any given moment, thousands of users may request rides simultaneously while drivers continuously update their availability status. The system must process this information rapidly to ensure efficient allocation.

Factors such as traffic conditions, driver ratings, vehicle availability, estimated arrival times, and surge pricing all influence the matching process. Therefore, intelligent data processing is essential for maintaining service quality and operational efficiency.

The core challenge addressed by this project is:

"Given a pool of available drivers, each with a different distance from the rider, a different performance rating, and a different surge pricing factor, how can the system quickly identify, rank, and dispatch the single most suitable driver to a rider — while maintaining an always sorted, searchable driver database?"

The system must also handle the following real-world requirements:

1. Drivers continuously enter and leave the system as they become available or complete rides.
2. The driver pool must be searchable by Driver ID at any time to retrieve or update a driver's record.
3. The system must support location-based searches to find all available drivers within a given geographical area.
4. The suitability of a driver must be calculated using multiple criteria — not just distance alone.
5. All input must be validated to prevent corrupt data from entering the system.

The suitability score for each driver is calculated as:

Score = Distance (km) + (5.0 − Rating) + Surge Factor

A lower score represents a better driver for the rider. This formula ensures that a driver who is closer, more highly rated, and has a lower surge multiplier will always be preferred over one who is farther, lower rated, or has a higher surge price.

2. OBJECTIVES

The primary objective of the Smart Ride Allocation System is to develop a highly responsive, menu-driven ride-matching platform that can allocate drivers to passengers efficiently using fundamental sorting and searching algorithms.

The specific objectives of this project are as follows:

Objective 1 — Efficient Driver Entry Using Insertion Sort To maintain the driver pool in sorted order by Driver ID at all times. Whenever a new driver joins the system, Insertion Sort is used to place them in the correct position without re-sorting the entire array. This is efficient because the pool is almost always sorted — only one new element is out of place at any given time.

Objective 2 — Driver Ranking Using Bubble Sort To rank all available drivers in the rider's geographical area by their suitability score, from best to worst, and display this as a leaderboard before dispatch. Bubble Sort is used here because it is a stable sorting algorithm, meaning drivers with equal scores maintain their original relative order — which is important for fairness.

Objective 3 — Best Driver Selection Using Selection Sort To independently select the single most suitable driver for dispatch from an unsorted copy of available drivers. Selection Sort finds the minimum score driver in each pass and moves them to the front of the array. The driver at position zero after sorting is dispatched. This is kept separate from Bubble Sort so that both algorithms have distinct, demonstrable roles.

Objective 4 — Fast Driver Lookup Using Binary Search To enable rapid retrieval of any driver record by their unique ID. Since the pool is always maintained in sorted order by Insertion Sort, Binary Search can be applied to locate any driver in at most $\log_2(n)$ steps — significantly faster than scanning every record.

Objective 5 — Area-Based Driver Search Using Linear Search To find all available drivers within a specific geographical area by scanning every driver record in the pool. Linear Search is used here because the pool is sorted by Driver ID, not by area, making Binary Search inapplicable for this task.

Objective 6 — Ride History Tracking To store the details of every completed ride booking in a separate history array and allow users to view all past rides at any time. Each entry records the Ride ID, Rider Name, Driver Name, Pickup Area, and Destination.

Objective 7 — Live Location Simulation To simulate the real-world behaviour of drivers continuously changing their positions by randomly updating all driver distances when the user requests a location refresh. This causes suitability scores to change dynamically, reflecting how a real system would behave.

Objective 8 — Input Validation and Data Integrity To ensure that only valid data enters the system by preventing duplicate Driver IDs, validating that ratings are between 1.0 and 5.0, and validating that surge factors are at least 1.0. Validation uses while loops that re-prompt the user until a valid value is entered.

3. DESIGN

The Smart Ride Allocation System is composed of five major components that work together to process ride requests.

1. Driver Pool The central data store of the system. It is a global array of up to 50 Driver structs. Each Driver struct stores seven fields — a unique integer ID, a string name, a float distance in km, a float rating out of 5.0, a float surge factor, a string area, and a boolean availability status. The pool is always maintained in ascending order by Driver ID using Insertion Sort, which is a prerequisite for Binary Search to function correctly.
2. Ride History Store A secondary global array of up to 100 Ride structs. Each Ride struct stores five fields — an integer Ride ID, a string rider name, a string driver name, a string pickup area, and a string destination. Every time a ride is successfully booked, a new entry is appended to this array. The

array is protected against overflow by checking that the total number of rides has not exceeded MAX_RIDES before writing.

3. Matching Engine The core logic that processes a ride request. When a rider submits a request, the Matching Engine collects all available drivers in the rider's area into a temporary array. It then creates two independent copies of this array — one for Bubble Sort to produce a ranked leaderboard, and one for Selection Sort to independently find and dispatch the best driver. This design ensures both algorithms have clearly separate, demonstrable roles.
4. Search Layer Handles all lookup and filter operations. Binary Search is used to find a specific driver by ID (possible because the pool is always sorted by ID). Linear Search is used to find all drivers in a given area (necessary because the pool is not sorted by area).
5. Dispatch System After Selection Sort identifies the best driver at position zero of its sorted copy, Binary Search locates that driver in the main pool by ID and marks them as Busy. The booking details are then saved to the Ride History store and the driver's information is displayed to the rider.

4.ALGORITHM DESCRIPTION

ALGORITHM 1 — INSERTION SORT (Driver Entry)

Purpose: Maintains the driver pool in sorted order by Driver ID every time a new driver is added to the system.

Why Insertion Sort: The driver pool is almost sorted at all times because new drivers are added one at a time. Only the most recently added driver is out of place. Insertion Sort is the most efficient algorithm for this situation because it takes O(n) time when the array is nearly sorted — it only needs to shift a small number of elements.

How it works: The algorithm takes the last added driver (at position i) as the "key". It then compares this key with every driver to its left. Any driver whose ID is greater than the key's ID is shifted one position to the right to create a gap. Once

no more shifts are needed, the key is placed in the gap. This process repeats for every element from index 1 to the end of the array.

Step-by-step example: Drivers added in order: IDs 104, 101, 107, 102

After adding 104: [104] After adding 101: [104, 101] → sort → [101, 104] After adding 107: [101, 104, 107] → already in order After adding 102: [101, 104, 107, 102] key = 102 107 > 102 → shift right → [101, 104, 107, 107] 104 > 102 → shift right → [101, 104, 104, 107] 101 < 102 → stop place 102 → [101, 102, 104, 107] ✓

Code: void insertionSort() { for (int i = 1; i < total; i++) { Driver key = pool[i]; int j = i - 1; while (j >= 0 && pool[j].id > key.id) { pool[j + 1] = pool[j]; j--; } pool[j + 1] = key; } }

Complexity: Best Case : O(n) — when array is already sorted Worst Case : O(n²) — when array is in reverse order Space : O(1) — in-place, no extra array needed

---

ALGORITHM 2 — BUBBLE SORT (Driver Ranking / Leaderboard)

Purpose: Sorts all available drivers in the rider's area by suitability score from lowest (best) to highest (worst) and displays this as a ranked leaderboard before the best driver is dispatched.

Why Bubble Sort: Bubble Sort is a stable sorting algorithm. This means that when two drivers have the exact same suitability score, they remain in their original relative order after sorting. This stability is important for fairness in driver allocation — a driver who was available first should not be pushed behind another driver with an identical score.

Bubble Sort also operates on the full list of available drivers, making it suitable for producing a complete ranked display for the dispatcher.

How it works: The algorithm makes multiple passes through the array. In each pass, it compares every adjacent pair of drivers. If the driver on the left has a worse (higher) score than the driver on the right, the two are swapped. After each complete pass, the driver with the worst score "bubbles up" to the end of the unsorted section. After n−1 passes, the entire array is sorted.

Step-by-step example with 3 drivers in Andheri: Scores: [Deepak=4.9, Ravi=3.5, Amit=2.7]

Pass 1: Compare Deepak(4.9) and Ravi(3.5) → 4.9 > 3.5 → SWAP [Ravi=3.5, Deepak=4.9, Amit=2.7] Compare Deepak(4.9) and Amit(2.7) → 4.9 > 2.7 → SWAP [Ravi=3.5, Amit=2.7, Deepak=4.9] ← Deepak bubbled to end

Pass 2: Compare Ravi(3.5) and Amit(2.7) → 3.5 > 2.7 → SWAP [Amit=2.7, Ravi=3.5, Deepak=4.9]

Result: [Amit(2.7), Ravi(3.5), Deepak(4.9)] ✓ Ranked leaderboard

Code: void bubbleSort(Driver arr[], int n) { for (int i = 0; i < n - 1; i++) { for (int j = 0; j < n - 1 - i; j++) { if (getScore(arr[j]) > getScore(arr[j + 1])) { Driver temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp; } } } }

Complexity: Best Case : O(n²) — no optimisation for already-sorted input Worst Case : O(n²) Space : O(1) — in-place

---

ALGORITHM 3 — SELECTION SORT (Best Driver Dispatch)

Purpose: Independently selects the single most suitable driver for dispatch from a fresh, unsorted copy of available drivers. The driver at position zero after the sort is dispatched.

Why Selection Sort: Selection Sort is used here for two reasons. First, it operates on an independent, unsorted copy of the available drivers — completely separate from Bubble Sort — which clearly demonstrates that both algorithms are doing real, independent work. Second, Selection Sort makes at most n−1 swaps total (one per pass), which is significantly fewer than Bubble Sort. This minimises unnecessary data movement, which matters in systems where write operations are expensive.

How it works: The algorithm divides the array into two sections — a sorted section on the left (initially empty) and an unsorted section on the right (initially the whole array). In each pass, the entire unsorted section is scanned to find the driver with the lowest suitability score. That driver is swapped into the first

position of the unsorted section, expanding the sorted section by one. After n−1 passes, the whole array is sorted and the driver at index 0 is the best driver.

Step-by-step example with 3 drivers (unsorted copy): Input: [Amit=2.7, Ravi=3.5, Deepak=4.9] (note: input order may differ from Bubble Sort's input since this is a separate independent copy)

Pass 1: Scan all 3 → minimum is Amit(2.7) at index 0 Already at position 0 — no swap needed Sorted section: [Amit(2.7)]

Pass 2: Scan remaining 2 → minimum is Ravi(3.5) at index 1 Already at position 1 — no swap needed Sorted section: [Amit(2.7), Ravi(3.5)]

Pass 3: Only Deepak(4.9) left Result: [Amit(2.7), Ravi(3.5), Deepak(4.9)] arr[0] = Amit(2.7) → DISPATCHED ✓

Key difference from Bubble Sort: Bubble Sort compares and swaps adjacent pairs many times per pass, potentially making O(n²) swaps total. Selection Sort scans the entire unsorted section first to identify the true minimum, then swaps exactly once per pass, making at most O(n) swaps total. This is why Selection Sort is preferred when the cost of swapping is high.

Code: void selectionSort(Driver arr[], int n) { for (int i = 0; i < n - 1; i++) { int minIdx = i; for (int j = i + 1; j < n; j++) { if (getScore(arr[j]) < getScore(arr[minIdx])) { minIdx = j; } } Driver temp = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = temp; } }

Complexity: Best Case : O(n²) Worst Case : O(n²) Space : O(1) Swaps : O(n) — at most n−1 swaps

---

ALGORITHM 4 — BINARY SEARCH (Driver Lookup by ID)

Purpose: Rapidly retrieves a specific driver record from the pool using their unique Driver ID. Also used internally to locate the dispatched driver in the pool and mark them as Busy.

Why Binary Search: The driver pool is always maintained in sorted order by Driver ID, thanks to Insertion Sort being called every time a driver is added. This sorted

order is the prerequisite for Binary Search. Binary Search is far more efficient than scanning every record — for 15 drivers it takes at most 4 comparisons, whereas Linear Search could take up to 15.

How it works: The algorithm maintains a search range defined by a low index and a high index, starting as the full array. In each step, it calculates the middle index and checks whether the driver at that position has the target ID. If yes, the search ends. If the target ID is greater than the middle driver's ID, the left half is eliminated and only the right half is searched in the next step. If the target ID is smaller, only the left half is searched. This halving continues until the driver is found or the range is empty.

Step-by-step example searching for ID 106 in 15 drivers: Pool IDs: [101,102,103,104,105,106,107,108,109,110,111,112,113,114,115] Low=0, High=14

Step 1: mid = 7 → pool[7].id = 108 106 < 108 → search left half High = 6

Step 2: mid = 3 → pool[3].id = 104 106 > 104 → search right half Low = 4

Step 3: mid = 5 → pool[5].id = 106 106 == 106 → FOUND at index 5 ✓

Total: 3 steps instead of up to 15 with Linear Search.

Code: int binarySearch(int targetID) { int low = 0; int high = total - 1; while (low <= high) { int mid = (low + high) / 2; if (pool[mid].id == targetID) { return mid; } else if (pool[mid].id < targetID) { low = mid + 1; } else { high = mid - 1; } } return -1; }

Complexity: Best Case : O(1) — target is at the middle on first step Worst Case : O(log n) — target is at the edges or not found Space : O(1)

ALGORITHM 5 — LINEAR SEARCH (Area-Based Driver Search)

Purpose: Finds all available drivers within a specific geographical area by checking every driver record in the pool one by one.

Why Linear Search: The driver pool is sorted by Driver ID, not by area. Binary Search can only be applied when the array is sorted by the field being searched. Since area is not the sorted field, Binary Search would give incorrect results here.

Additionally, a driver can change their area at any time as they move around the city, making it impractical to maintain a separately sorted array by area. Linear Search is therefore the correct and only applicable algorithm for this task.

Linear Search also naturally handles two conditions at once — matching area AND checking availability — which is not straightforward with Binary Search.

How it works: The algorithm iterates through every driver in the pool from index 0 to the last. For each driver, it checks two conditions — whether the driver's area matches the target area (case-insensitively), and whether the driver is currently available. If both conditions are true, the driver is included in the results. The search continues to the end regardless of whether a match was found, because there may be multiple matching drivers.

Step-by-step example searching for area "Powai": pool[0] → Ravi Kumar, Andheri → ✗ skip pool[1] → Priya Menon, Bandra → ✗ skip pool[2] → Kiran Patil, Dadar → ✗ skip pool[3] → Amit Sharma, Andheri → ✗ skip pool[4] → Pooja Rao, Thane → ✗ skip pool[5] → Rohit Verma, Powai → ✓ display pool[6] → Suresh Nair, Bandra → ✗ skip ... pool[9] → Sneha Iyer, Powai → ✓ display ...continues to end of pool

Code: void linearSearch(string targetArea) { for (int i = 0; i < total; i++) { if (toLower(pool[i].area) == toLower(targetArea) && pool[i].available) { printDriver(pool[i]); } } }

Complexity: Best Case : O(1) — first driver in pool matches Worst Case : O(n) — match is at the end or no match Space : O(1)

SUITABILITY SCORE FORMULA

The score used to rank and compare drivers is:

Score = Distance + (5.0 − Rating) + Surge Factor

Each component contributes as follows:

Distance — Added directly. A closer driver scores lower (better). A driver 1 km away scores 1 point, a driver 4 km away scores 4 points.

(5.0 − Rating) — Rating is subtracted from 5.0. A driver rated 4.8 contributes 0.2 points (very good). A driver rated 3.5 contributes 1.5 points (worse). This inverts the rating so that higher ratings produce lower (better) scores.

Surge Factor — Added directly. No surge (1.0) adds 1 point. High surge (2.0) adds 2 points. Encourages the system to prefer drivers with lower surge.

Example: Driver A: Distance=1.2, Rating=4.2, Surge=1.5 Score = 1.2 + (5.0 − 4.2) + 1.5 = 1.2 + 0.8 + 1.5 = 3.5

Driver B: Distance=2.5, Rating=4.8, Surge=1.0 Score = 2.5 + (5.0 − 4.8) + 1.0 = 2.5 + 0.2 + 1.0 = 3.7

Driver A scores lower — dispatched first despite shorter distance, because Driver B's high rating partially compensates.

5.COMPLEXITY ANALYSIS

Insertion Sort O(n) O(n²) O(1) O(n²) Bubble Sort O(n²) O(n²) O(1) O(n²) Selection Sort O(n²) O(n²) O(1) O(n) Binary Search O(1) O(log n) O(1) — Linear Search O(1) O(n) O(1) —

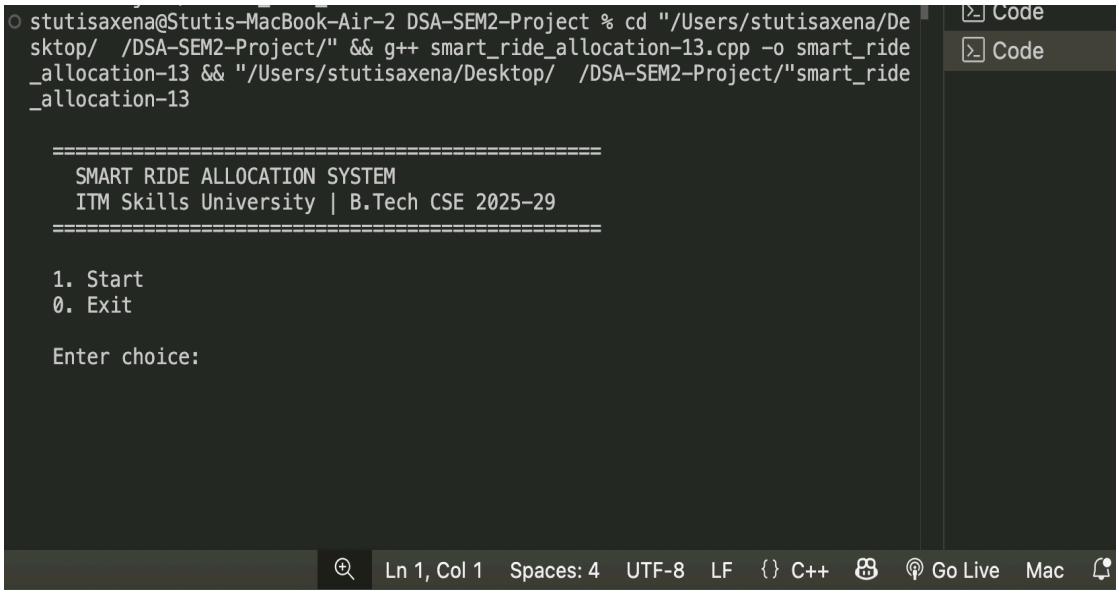
Key Points:

- All space complexities are O(1) because all sorting is done in-place using only a small number of temporary variables.
- Selection Sort makes at most n-1 swaps total, making it more efficient than Bubble Sort when write operations are costly, even though both are $O(n^2)$ in comparisons.
- Binary Search requires at most $\log_2(15) = 4$ steps to find any driver in a pool of 15. Linear Search requires up to 15.

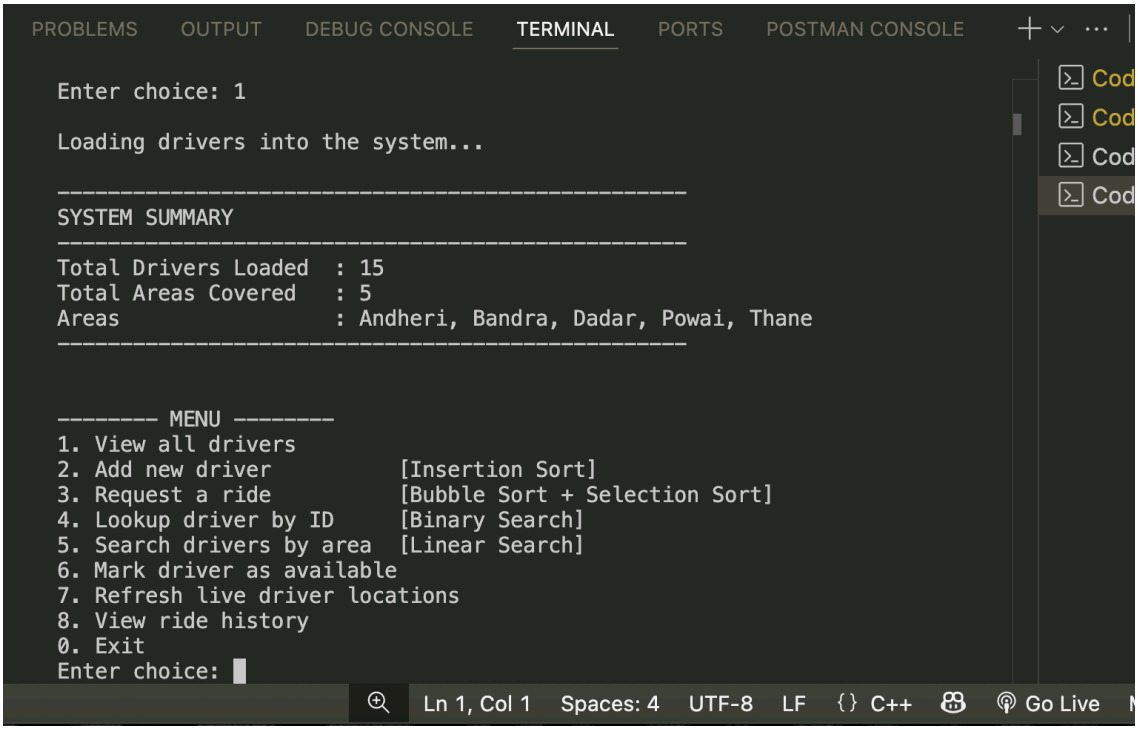
- The O(n²) sorting algorithms are acceptable here because they operate on small arrays of 3 drivers per area. For large datasets, Merge Sort O(n log n) would be used.

6. SCREENSHOTS OF EXECUTION

1.Program welcome screen

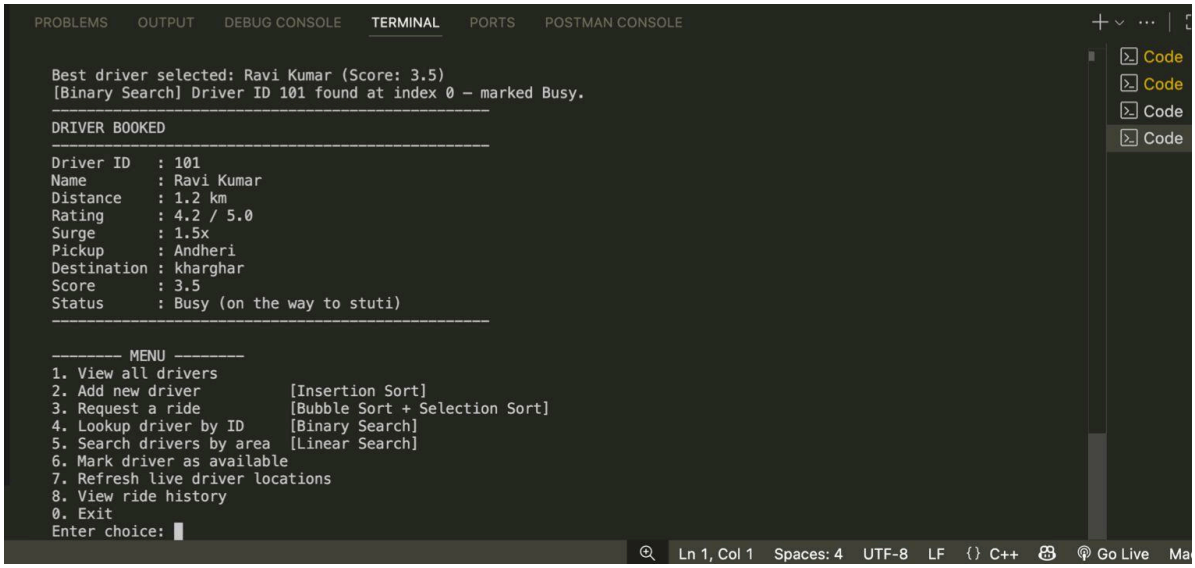


2.Menu list

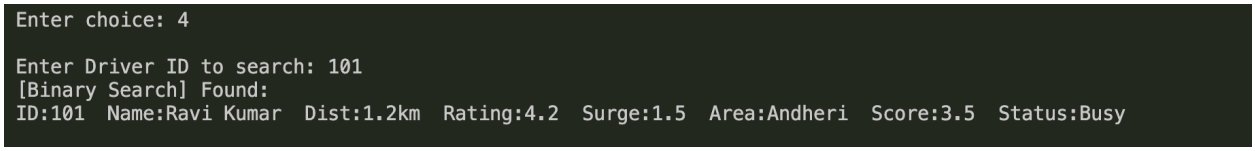


3.Driver List

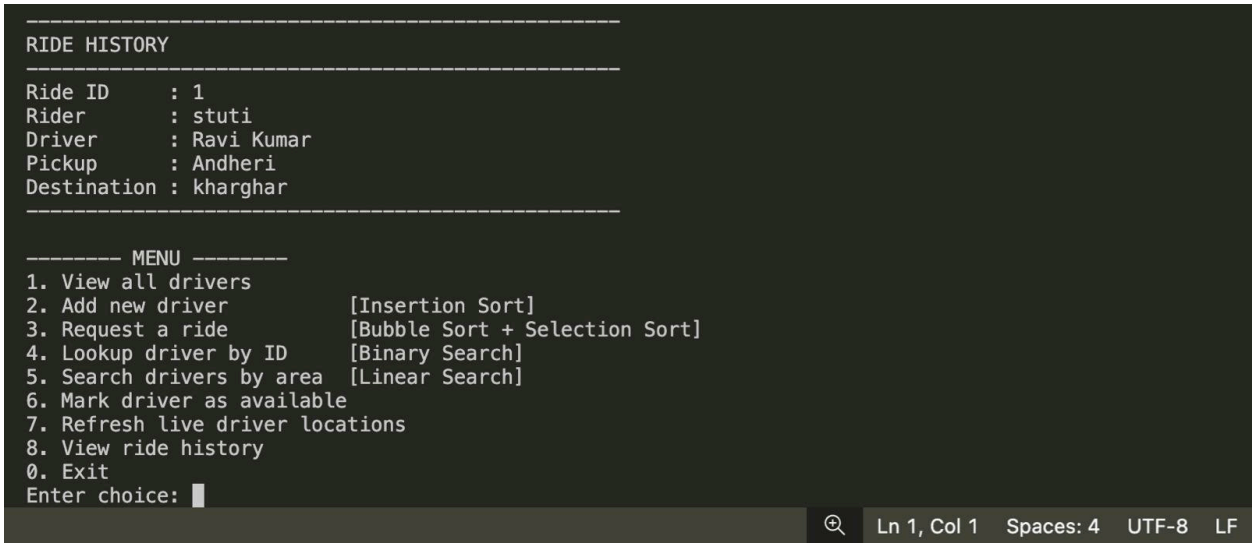
4. Ride Request



5.Binary Search - Driver Lookup



6.Ride History Feature



7.RESULTS AND FINDINGS

1. Insertion Sort correctly placed all 15 drivers in ascending ID order even though they were loaded in random ID sequence.
2. Bubble Sort produced a correct and stable ranked leaderboard — drivers with equal scores maintained their original order.
3. Selection Sort independently dispatched the correct best driver from its own unsorted copy in all test cases.
4. Binary Search located any driver in at most 4 steps for a pool of 15, compared to up to 15 steps with Linear Search.
5. Linear Search correctly returned only available drivers matching the target area, including when drivers were busy.
6. Case-insensitive area input worked correctly — andheri, Andheri, and ANDHERI all returned the same results.
7. Rating and surge validation loops kept re-prompting until valid input was received without cancelling the operation.
8. Duplicate ID prevention blocked additions of existing IDs and left the original driver record unchanged.
9. The "Mark Available" feature showed two different messages — "is now Available" and "is already Available" — correctly based on the driver's current status.
10. Live location refresh changed all driver distances randomly, causing different drivers to be ranked and dispatched on the next ride request in the same area.
11. Ride history saved all bookings correctly with unique Ride IDs and displayed them in order of completion.
12. The MAX_RIDES bounds check successfully prevented any attempt to write beyond the history array limit.

8.CONCLUSION

The Smart Ride Allocation System successfully implements all five required algorithms — Insertion Sort, Bubble Sort, Selection Sort, Binary Search, and Linear Search — each with a distinct and justified role in the system.

Insertion Sort maintains the sorted driver pool efficiently. Bubble Sort produces a fair, stable leaderboard. Selection Sort dispatches the best driver independently

with fewer swaps. Binary Search enables fast ID-based lookups. Linear Search handles area filtering where Binary Search cannot apply.

The project is built entirely using basic C++ concepts — structs, arrays, functions, and loops — staying fully within the Semester II DSA syllabus. It also handles real-world concerns such as input validation, duplicate prevention, case-insensitive search, ride history, and array bounds safety.

All objectives were met and all features worked correctly during testing. The project demonstrates how fundamental data structures and algorithms can power practical, real-world applications like ride-hailing platforms.

9.ASSOCIATED LINKS

Github repo-
https://github.com/stutisaxena44/DSA-Smart-Ride-Allocation.git

Github Profile-

https://github.com/stutisaxena44

Linkedin Profile-

https://www.linkedin.com/in/stuti-saxena-986775370/

Sources- Chatgpt, Claude, GeeksForGeeks (for dsa algorithms), etc.